Time Complexity 01

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Measuring Complexity

- A **solvable** problem takes time to solve
  - Some problems may take too long to solve
- We are going to measure the time to solve a problem by an algorithm
  - An algorithm will be represented by a TM
  - Count the maximum number of steps to process an input of length n until it halts
- Let M be a deterministic Turing machine that halts on all inputs
  - The **running time** or **time complexity** of M is the function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps that M uses on any input of length n
  - If $f(n)$ is the running time of M, M runs in time $f(n)$
    - M is an $f(n)$ time Turing machine
  - Customarily we use n to represent the length of the input

Big-O and Small-o Notations

- Given a TM M, its running time $f(n)$ may look like the following:
$$f(n) = 3n^3 + 20n^2 + 15n + 12$$
- To eliminate the complexity of a running time of a Turing machine, we estimate it
- This is called **asymptotic analysis**
  - Consider only when the Turing machine runs on large input
  - Consider only the highest order term
  - Discard the coefficient
- We generally use Big-O and Small-o notations

Big-O Definition

- Let f and g be functions $f, g : \mathbb{N} \to \mathbb{R}^+$
- We say that $f(n) = \mathcal{O}(g(n))$ or $f(n)$ is $\mathcal{O}(g(n))$ if positive integers c and n_0 exist such that for every integer $n \geq n_0$,

$$f(n) \leq cg(n)$$

- When $f(n) = \mathcal{O}(g(n))$:
  - $g(n)$ is an **upper bound** for $f(n)$, or
  - More precisely, $g(n)$ is an **asymptotic upper bound** for $f(n)$

Big-O Examples

- Show that $f(n) = 6n^3 + 2n^2 + 20n + 45$ is $\mathcal{O}(n^3)$
- Need to find c and n_0 such that $f(n) \leq cn^3$ for $n \geq n_0$
  - We know the following:
    - $2n^2 \leq 2n^3$ for $n \geq 1$
    - $20n \leq 20n^3$ for $n \geq 1$
    - $45 \leq 45n^3$ for $n \geq 1$
  - Thus, we have

$$\begin{aligned} f(n) = 6n^3 + 2n^2 + 20n + 45 &\leq 6n^3 + 2n^3 + 20n^3 + 45n^3 \\ &= (6 + 2 + 20 + 45)n^3 \\ &= 73n^3 \text{ for } n \geq 1 \end{aligned}$$

  - In other words,

$$f(n) \leq 73n^3 \quad \text{for } n \geq 1$$

  - From the above equation, what are c and n_0?
    - $c = 73$
    - $n_0 = 1$
  - Thus, $6n^3 + 2n^2 + 20 + 45$ is $\mathcal{O}(n^3)$

Big-O Examples

- Show that $f(n) = 2n \log_2 n + 20n + 10$ is $\mathcal{O}(n \log_2 n)$
- Need to find c and n_0 such that $f(n) \leq c(n \log_2 n)$ for $n \geq n_0$
  - We know the following:
    - $20n \leq 20n \log_2 n$ for $n \geq 1$
    - $10 \leq 10n \log_2 n$ for $n \geq 1$
  - Thus, we have

$$\begin{aligned} f(n) = 2n \log_2 n + 20n + 10 &\leq 2n \log_2 n + 20n \log_2 n + 10n \log_2 n \\ &= (2 + 20 + 10) n \log_2 n \\ &= 32n \log_2 n \text{ for } n \geq 1 \end{aligned}$$

  - In other words,

$$f(n) \leq 32n \log_2 n \quad \text{for } n \geq 1$$

  - From the above equation, what are c and n_0?
    - $c = 32$
    - $n_0 = 1$
  - Thus, $2n \log_2 n + 20n + 10$ is $\mathcal{O}(n \log_2 n)$

Big-O Examples

- Upper bounds of the form n^c where $c > 0$ is called **polynomial bounds**
  - $\mathcal{O}(n^2)$
  - $\mathcal{O}(n^{10})$
  - $\mathcal{O}(n \log_2 n)$ (bounded by $\mathcal{O}(n^2)$)
- Upper bounds of the form $2^{(n^\delta)}$ is called **exponential bounds**
  - $\mathcal{O}(2^n)$
  - $\mathcal{O}(3^n)$
    - $3^n = (2^{\log_2 3})^n = 2^{\log_2 3 \times n}$ and $\log_2 3$ is a constant

Small-o Notation

Definition 7.5

Let f and g be functions $f, g : \mathbb{N} \to \mathbb{R}^+$. Say that $f(n) = o(g(n))$ if

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

In other words, $f(n) = o(g(n))$ means that for any real number $c > 0$, a number n_0 exists, where $f(n) < cg(n)$ for all $n \geq n_0$.

- Simply strictly less than
  - $\sqrt{n} = o(n)$
  - $n = o(n \log(\log n))$
  - $n \log(\log n) = o(n \log n)$
  - $n \log n = o(n^2)$
  - $n^2 = o(n^3)$

Analyzing Algorithms

- Consider a Turing machine M_1 that decides

$$A = \{0^k 1^k \mid k \geq 0\}$$

$M_1 =$ "On input string w:

1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
2. Repeat if both 0s and 1s remain on the tape:
3. Scan across the tape, crossing off a single 0 and a single 1.
4. If 0s still remain after all the 1s have been crossed off, or if 1s still remain after all the 0s have been crossed off, *reject*. Otherwise, if neither 0s nor 1s remain on the tape, *accept*."

- Find $f(n)$, the maximum number of steps for M_1 to process a string w of length n
  - There are multiple input strings of length n
  - The number of steps of an input of length n depends on the input pattern

Analyzing Algorithm

- Consider all possible inputs of length 4:

Input	Step that M_1 halts
0000	4 (0 iterations)
0001	4 (1 iterations)
0010	1
0011	4 (2 iterations)
0100	1
0101	1
0110	1
0111	4 (1 iterations)
1000	1
1001	1
1010	1
1011	1
1100	1
1101	1
1110	1
1111	4 (0 iterations)

- The maximum number of steps on an input of length n occurs when $w = 0^k1^k$ where $k = \frac{n}{2}$

Analyzing Algorithms

1. Scan across the tape and *reject* if a 0 is found to the right of a 1
   - This step requires TM M_1 to move its tape head all the way to the right until it encounters a blank symbol and move its tape head all the way to the left-most square
   - This step requires either n or $2n$ steps
   - In other words, this step is $\mathcal{O}(n)$
2. Repeat if both 0s and 1s remain on the tape:
3. Scan across the tape, crossing off a single 0 and a single 1.
   - Step 3 requires n steps or $\mathcal{O}(n)$
     - Cross the first 0, move the tape head $n/2$ step to the first 1, cross it off and move the tape head $n/2$ step to position on top of the first of the remaining 0s
   - There will be $n/2$ 0s and $n/2$ 1s
     - The step 3 is repeated $n/2$ times
   - Steps 2 and 3 together is $n/2 \times \mathcal{O}(n)$ or $\mathcal{O}(n^2)$.

Analyzing Algorithms

4. If 0s still remain after all the 1s have been crossed off, or if 1s still remain after all the 0s have been crossed off, $reject$. Otherwise, if neither 0s nor 1s remain on the tape, $accept$.
   - Simply scan the whole tape looking for a 0 or a 1
   - This step can be done in n step
   - This step is $\mathcal{O}(n)$

- The running time of M_1 is

$$\mathcal{O}(n) + \mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2 + 2n) = \mathcal{O}(n^2)$$

Time Complexity Class

Definition 7.7

Let $t : \mathbb{N} \to \mathbb{R}^+$ be a function. Define the **time complexity class**, $\mathrm{TIME}(t(n))$, to be the collection of all languages that are decidable by an $\mathcal{O}(t(n))$ time Turing machine.

- From the previous language $A = \{0^k 1^k \mid k \geq 0\}$ with TM M_1

$$A \in \mathrm{TIME}(n^2)$$

- Can we construct a TM M_2 such that $f(n) = o(n^2)$?
  - Asymptotically better than $\mathcal{O}(n^2)$
- What if we cross off two 0s and two 1s in step 3?
  - Practically faster but not asymptotically better
  - Step 3 will be repeated $n/4$ times but $n/4 \times \mathcal{O}(n) = \mathcal{O}(n^2)$

Asymptotically Faster for Deciding A

- Consider TM M_2 that decides $A = \{0^k 1^k \mid k \geq 0\}$:

  $M_2 =$ "On input string w:

  1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
  2. Repeat as long as some 0s and some 1s remain on the tape.
  3. Scan across the tape, checking whether the total number of 0s and 1s remaining is even or odd. If it is odd, *reject*.
  4. Scan again across the tape, crossing off every other 0 starting with the first 0, and then crossing off every other 1 starting with the first 1.
  5. If no 0s and no 1s remain on the tape, *accept*. Otherwise, *reject*.

Asymptotically Faster for Deciding A

- Step 1 and 5 are $\mathcal{O}(n)$ as in M_1
- Step 3, to scan and check whether the number of 0s and 1s are even or odd is $\mathcal{O}(n)$
- In step 4, to cross off every other 0s and 1s is $\mathcal{O}(n)$
  - Every time this step is executed, the number of 0s and 1s are cut in half
  - If there are $n/2$ 0s and $n/2$ 1s, step 4 needs to be repeated $1 + \log_2(n/2)$ times
  - For example ($n = 16$), $8 \to 4 \to 2 \to 1 \to 0$ (cut in half 4 times) for input $0^8 1^8$
- Steps 2, 3 and 4 all together is

$$(1 + \log_2(n/2)) \times (\mathcal{O}(n) + \mathcal{O}(n)) = \mathcal{O}(n \log n)$$

- Thus, TM M_2 run-time is

$$\mathcal{O}(n) + \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n)$$

Deciding A in Linear Time

- The language A can be decided in $\mathcal{O}(n)$
  - But we need the second tape
- Two-tape TM M_3 decides A in $\mathcal{O}(n)$

  $M_3 =$"On input string w:

  1. Scan across tape 1 and *reject* if a 0 is found to the right of a 1.
  2. Scan across the 0s on tape 1 until the first 1. At the same time, copy the 0s onto tape 2.
  3. Scan across the 1s on tape 1 until the end of the input. For each 1 read on tape 1, cross off a 0 on tape 2. If all 0s are crossed off before all the 1s are read, *reject*.
  4. If all the 0s have now been crossed off, *accept*. If any 0s remain, *reject*."
- Run-time of each step is $\mathcal{O}(n)$
  - TM M_3 runs in $\mathcal{O}(n)$
- The run-time complexity of the language A depends on the model of computation

Complexity Relationships Among Models

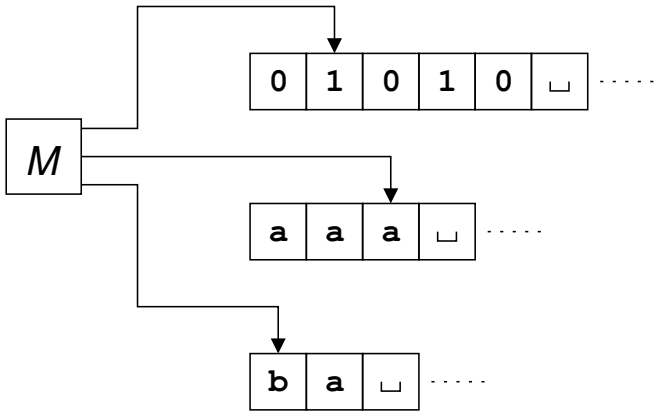
Theorem 7.8

Let $t(n)$ be a function where $t(n) \geq n$. Then every $t(n)$ time multitape Turing machine has an equivalent $\mathcal{O}(t^2(n))$ time single-tape Turing machine.

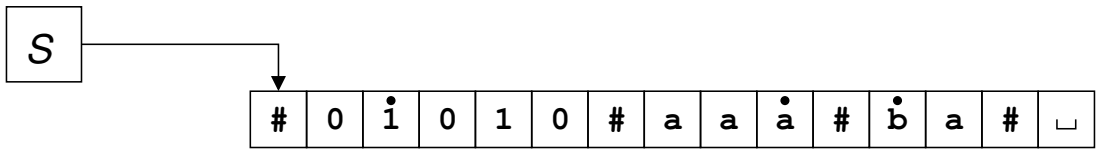
- Suppose we have a language A that can be decided by a multitape Turing machine M in $t(n)$ steps where n is the length of an input string
- The above theorem says that there exists a single-tape TM that can decide A in $\mathcal{O}(t^2(n))$
  - $t^2(n) = t(n) \times t(n)$
- To prove the above theorem, we simply need to construct a single tape TM that decides A in $\mathcal{O}(t^2(n))$
- But what is the language A?
  - Since we do not know what the language, let's try to simply simulate multitape TM using single-tape TM

Complexity Relationships Among Models

- **Recall** that a **multitape** TM is a k-tape TM where its input appears on tape 1



- We can simulate multitape TM using a single-tape TM as shown below:



- If TM M halts in $t(n)$ steps, how many step does TM S needs in terms of $\mathcal{O}$?

Proof of Theorem 7.8

- Let M be a k-tape TM for some $k \geq 2$ that runs in $t(n)$ on input string of length n
- We are going to construct a single-tape TM S that simulate multitape TM M
- The first step is for S to copy contents of all M's k tapes into its tape
  - Note that contents of all M's k tapes are fixed except the first one (input tape)
  - Suppose the length of strings on tape 2 to tape k be a constant c and the length of input string is n
  - It requires S to perform $c + n + (k + 1)$ steps to copy
    - The $k + 1$ steps are from writing # symbols
  - Since c and k are constant, this step is $\mathcal{O}(n)$
- The next step is to simulate M's steps

Proof of Theorem 7.8

- For each step of M, first S needs to know symbols under M's tape heads
  - S needs to scan and remember all k-dot symbols
  - Since M performs $t(n)$ steps, the maximum length of an active portion of each tape is $t(n)$
    - If the length of the content of a tape is $t(n) + 1$, it requires TM M $t(n)$ step to reach the last symbol
    - By the time it gets to the last symbol, TM M halts
  - S needs $k \times t(n)$ steps to scan for all k-dot symbols
  - Scanning for all dot symbols is $\mathcal{O}(t(n))$

Proof of Theorem 7.8

- Next S needs to update all k-dot symbol
  - Sometime, M may move a tape head to the right over a blank symbol
  - But for TM S, it will be on the # symbol
    - In this case, S needs to shift a portion of its own tape one cell to the right
    - If it is M's first tape head, it requires to shift the content of $k-1$ tapes

      Require $(k-1) \times t(n)$ steps for S to shift

      where $t(n)$ is the maximum length of an active portion of each tape
    - If it is M's second tape head, it requires to shift the content of $k-2$ tapes

      Require $(k-2) \times t(n)$ steps for S to shift
    - If all need to be shifted, it requires $\frac{k(k-1)}{2} \times t(n)$ steps
  - This step is $\mathcal{O}(t(n))$ since $\frac{k(k-1)}{2}$ is a constant
- To simulate one step of M, it is

$$\text{scan for dots} + \text{update dots} = \mathcal{O}(t(n)) + \mathcal{O}(t(n)) = \mathcal{O}(t(n))$$

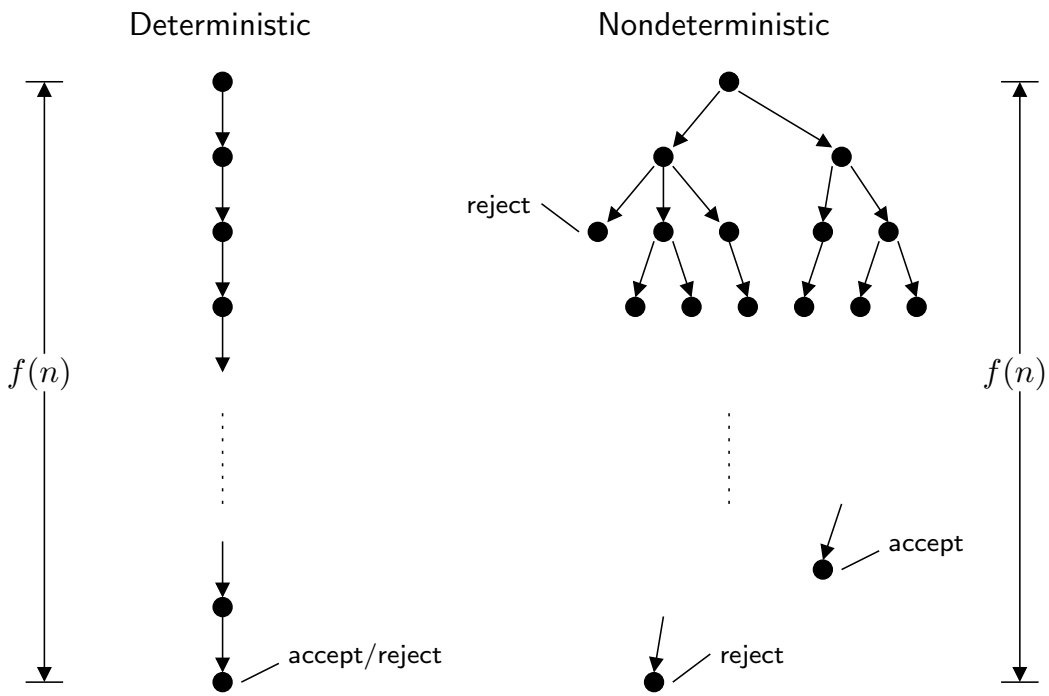
Proof of Theorem 7.8

- Note that M performs $t(n)$ steps
- Thus, for S to simulate M for $t(n)$ steps, it needs to perform at most
$$t(n) \times \mathcal{O}(t(n)) = \mathcal{O}(t^2(n))$$
- Therefore, the total run-time is
$$\mathcal{O}(n) + \mathcal{O}(t^2(n)) = \mathcal{O}(t^2(n))$$
where
  - $\mathcal{O}(n)$ steps to copying M's tape contents, and
  - $\mathcal{O}(t^2(n))$ steps to simulate M's $t(n)$ steps
- If a multitape TM M decides a language A in $t(n)$ steps, we can simulate TM M using a single tape TM S which can decide A in $\mathcal{O}(t^2(n))$

Nondeterministic Turing Machine

Definition 7.9

Let N be a nondeterministic Turing machine that is a decider. The **running time** of N is the function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps that N uses on any branch of its computation on any input of length n.



Nondeterministic Turing Machine

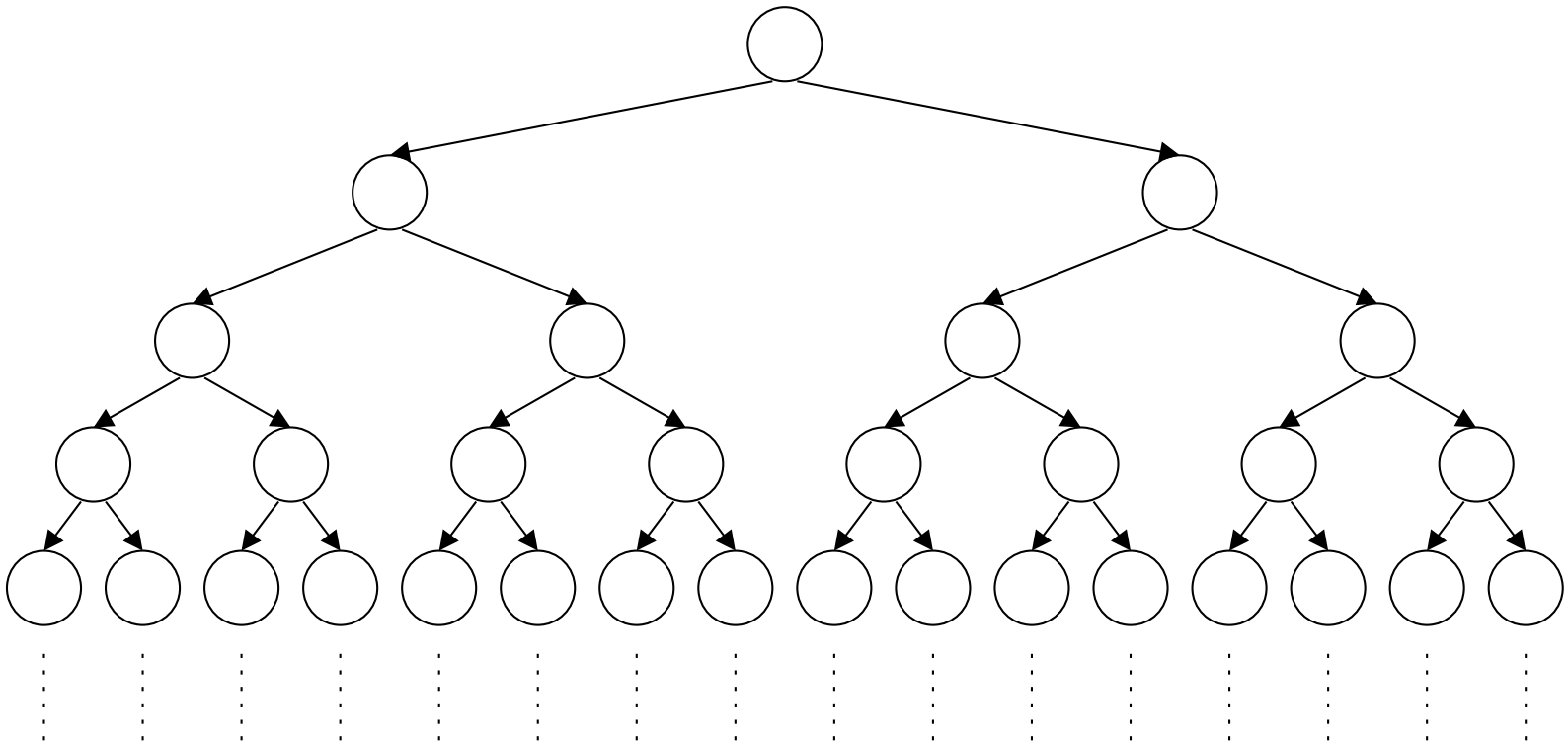
Theorem 7.11

Let $t(n)$ be a function, where $t(n) \geq n$. Then every $t(n)$ time nondeterministic single-tape Turing machine has an equivalent $2^{\mathcal{O}(t(n))}$ time deterministic single-tape Turing machine.

- Recall the nondeterministic Turing machine
  - The computation is a tree
  - The machine accepts as soon as one of its branch enter the accept state
  - The machine rejects if all of its branch are in the reject state
- Since we have to consider the maximum number of steps on any branch, we have to consider the longest number of steps for the machine to accept or reject an input string of length n

Computation Tree

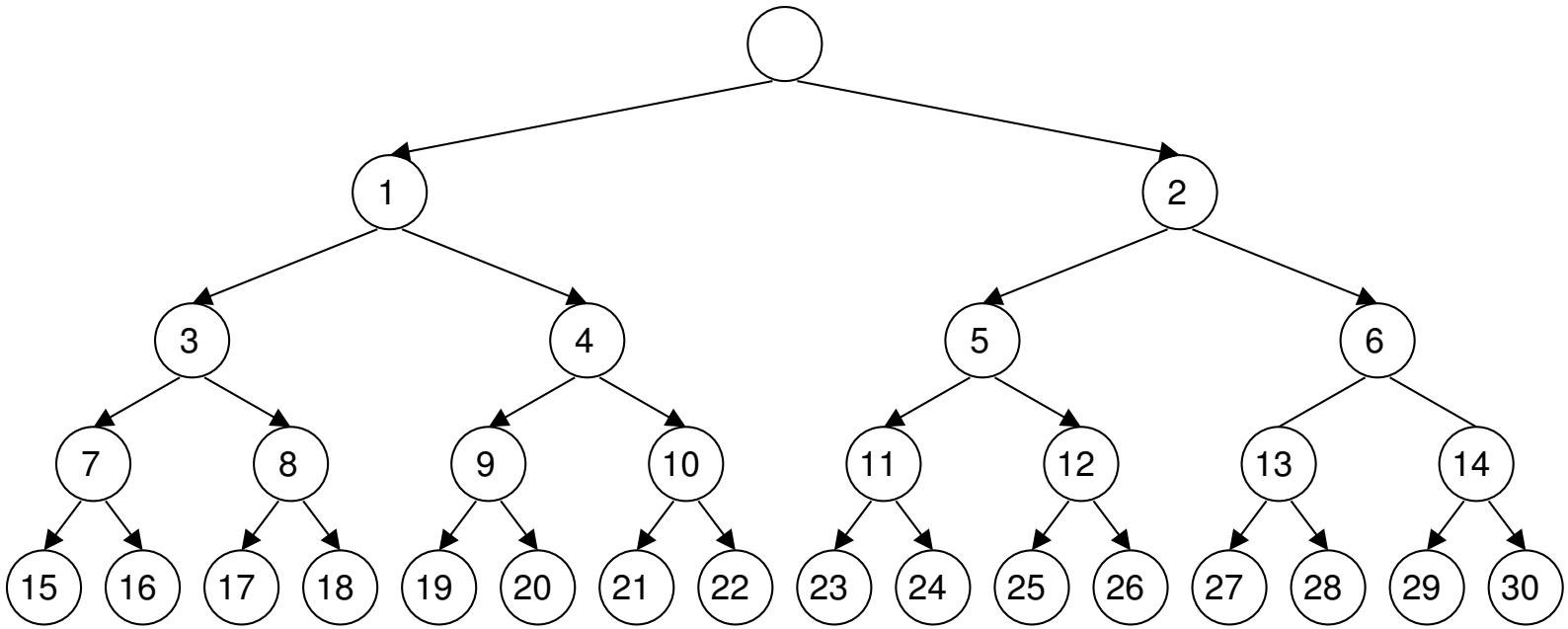
- The following is an example of a computation tree of a nondeterministic Turing machine where at each state, it can split to at most two machines.



- This can be applied to NTM that splits to at most k copies

Computation Tree

- We can not simulate one branch at a time
  - The computation of each branch may not end (enter infinite loop)
  - No chance to simulate the next branch
  - But NTM may accept the input string because another branch enters the accept state
- We need to simulate like **breadth first search**



Computation Tree

- Recall that this NTM runs in $t(n)$ steps
  - The height of its computational tree will be $t(n)$
  - The number of steps to reach is node is at most $t(n)$
- The number of nodes in a full binary tree of height h is $2^h - 1$
- Since the height of its computational tree is $t(n)$, the number of nodes is $2^{t(n)} - 1$

Computational Tree

- Thus, the total run-time is

$$t(n) \times 2^{t(n)} = 2^{\mathcal{O}(t(n))}$$

- This is the run-time of multitape Turing machine
- For a single tape TM, the run-time is the square of multitape TM
- Thus, the run-time is

$$(2^{\mathcal{O}(t(n))})^2 = 2^{2\mathcal{O}(t(n))} = 2^{\mathcal{O}(2t(n))} = 2^{\mathcal{O}(t(n))}$$